

```
# data sources:https://www.kaggle.com/datasets/adarshsng/lending-club-loan-data-csv
```

Double-click (or enter) to edit

```
from google.colab import files
files.upload()
```



选择文件

 kaggle.json

- kaggle.json(application/json) – 74 bytes, last modified: 2025/6/20 – 100% done

Saving kaggle.json to kaggle.json

```
{'kaggle.json': b'{"username":"lingyiwu6igmailcom","key":"9a07ea8763284ce3e6afbc6e7d4f0189"}'}
```

```
import os
os.makedirs("/root/.kaggle", exist_ok=True)
!mv kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json
```

```
!kaggle datasets download -d adarshsng/lending-club-loan-data-csv
```

 Dataset URL: <https://www.kaggle.com/datasets/adarshsng/lending-club-loan-data-csv>

License(s): DbCL-1.0

Downloading lending-club-loan-data-csv.zip to /content

91% 308M/339M [00:01<00:00, 198MB/s]

100% 339M/339M [00:01<00:00, 255MB/s]

```
import zipfile
with zipfile.ZipFile("/content/lending-club-loan-data-csv.zip","r") as Lending_club:
    Lending_club.extractall("lending-club-loan-data-csv")
```

```
import zipfile
with zipfile.ZipFile("/content/lending-club-loan-data-csv.zip","r") as Lending_club:
    Lending_club.extractall("lending-club-loan-data-csv")
```

```
from google.colab import drive
drive.mount("/content/drive")
```

 Mounted at /content/drive

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from tabulate import tabulate
```

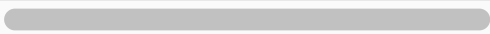
```
import pandas as pd
lending_club = pd.read_csv("/content/lending-club-loan-data-csv/loan.csv")
lending_club.head()
```

 /tmp/ipython-input-46-1239543633.py:2: DtypeWarning: Columns (19,47,55,112,123,124,125,128,129,130,133,139,140,141) have r

```
lending_club = pd.read_csv("/content/lending-club-loan-data-csv/loan.csv")
```

	id	member_id	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	sub_grade	...	hardship_p
0	NaN	NaN	2500	2500	2500.0	36 months	13.56	84.92	C	C1	...	
1	NaN	NaN	30000	30000	30000.0	60 months	18.94	777.23	D	D2	...	
2	NaN	NaN	5000	5000	5000.0	36 months	17.97	180.69	D	D1	...	
3	NaN	NaN	4000	4000	4000.0	36 months	18.94	146.51	D	D2	...	
4	NaN	NaN	30000	30000	30000.0	60 months	16.14	731.78	C	C4	...	

5 rows x 145 columns



```
lending_club.isnull().sum()
```



	0
id	2260668
member_id	2260668
loan_amnt	0
funded_amnt	0
funded_amnt_inv	0
...	...
settlement_status	2227612
settlement_date	2227612
settlement_amount	2227612
settlement_percentage	2227612
settlement_term	2227612

145 rows × 1 columns

dtype: int64

```
thresh = len(lending_club) * 0.7
lending_club = lending_club.dropna(thresh=thresh, axis=1)
lending_club = lending_club.dropna()
lending_club.head()
```



	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	sub_grade	emp_title	emp_length	...	pe
0	2500	2500	2500.0	36 months	13.56	84.92	C	C1	Chef	10+ years	...	
1	30000	30000	30000.0	60 months	18.94	777.23	D	D2	Postmaster	10+ years	...	
2	5000	5000	5000.0	36 months	17.97	180.69	D	D1	Administrative	6 years	...	
3	4000	4000	4000.0	36 months	18.94	146.51	D	D2	IT Supervisor	10+ years	...	
4	30000	30000	30000.0	60 months	16.14	731.78	C	C4	Mechanic	10+ years	...	

5 rows × 87 columns

lending_club.nunique()



0

loan_amnt	1561
funded_amnt	1561
funded_amnt_inv	1603
term	2
int_rate	356
...	...
total_bc_limit	9777
total_il_high_credit_limit	186578
hardship_flag	2
disbursement_method	2
debt_settlement_flag	2

87 rows × 1 columns

dtype: int64

lending_club['loan_status'].value_counts()



count

loan_status	
Fully Paid	764818
Current	671801
Charged Off	190439
Late (31–120 days)	15965
In Grace Period	6768
Late (16–30 days)	2635
Default	25

dtype: int64

```
import xgboost as xgb
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
```

```
lending_club = lending_club.copy()
y=lending_club["loan_status"]
x= lending_club.drop(columns = ["loan_status"])
le = LabelEncoder()
```

```
for col in x.select_dtypes(include='object').columns:
    x[col] = LabelEncoder().fit_transform(x[col].astype(str))
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

```
le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train)
model = xgb.XGBClassifier(max_depth=6,n_estimators = 100,use_label_encoder=False,eval_metric = "logloss")
model.fit(x_train, y_train_encoded)
```

```
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [03:08:00] WARNING: /workspace/src/learner.cc:74:
Parameters: { "use_label_encoder" } are not used.
```

```
warnings.warn(msg, UserWarning)
```

XGBClassifier

XGBClassifier(base_score=None, booster=None, callbacks=None, colsample_bylevel=None, colsample_bynode=None, colsample_bytree=None, device=None, early_stopping_rounds=None, enable_categorical=False, eval_metric='logloss', feature_types=None, gamma=None, grow_policy=None, importance_type=None, interaction_constraints=None, learning_rate=None, max_bin=None, max_cat_threshold=None, max_cat_to_onehot=None, max_delta_step=None, max_depth=6, max_leaves=None, min_child_weight=None, missing=nan, monotone_constraints=None, multi_strategy=None, n_estimators=100, n_jobs=None, num_parallel_tree=None, objective='multi:softprob', ...)

```
import matplotlib.pyplot as plt
xgb.plot_importance(model, max_num_features=25, importance_type='gain')
plt.show()
```

